

Architecture Overview

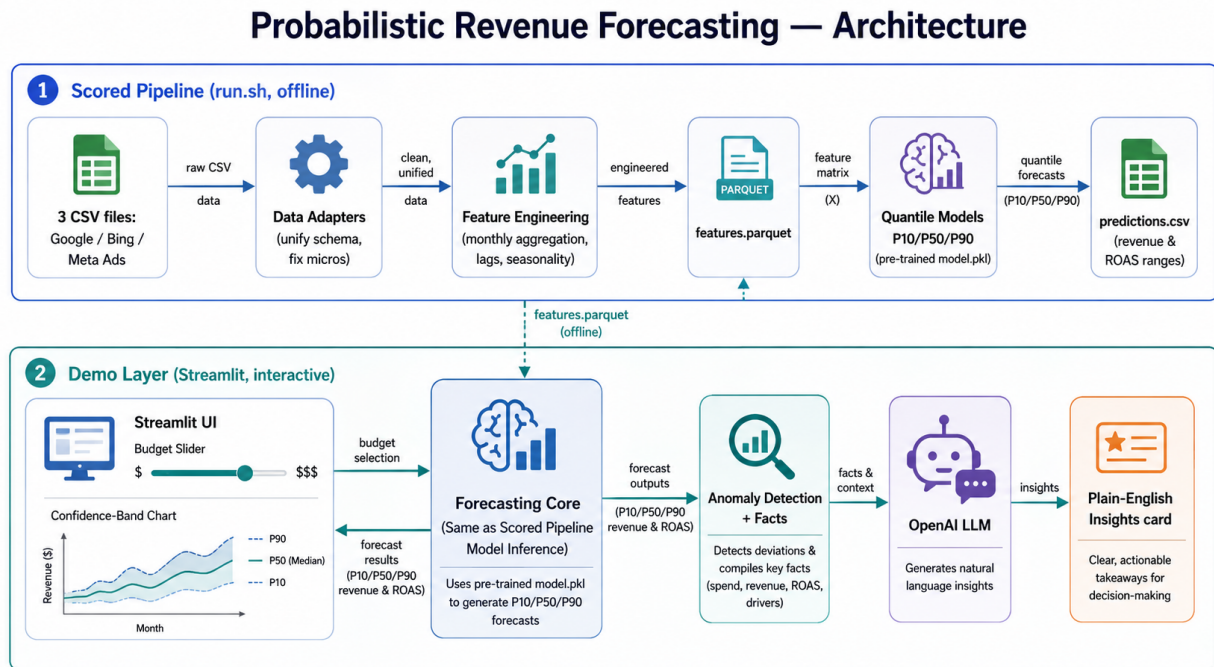


Figure 1: System architecture

Stack

Layer	Technology
Language	Python 3.11
Data / compute	pandas, numpy, pyarrow
Modeling	scikit-learn (GradientBoostingRegressor, quantile loss), joblib
Frontend / demo	Streamlit + Plotly
LLM integration	OpenAI API (via <code>openai</code>), config via <code>python-dotenv</code>

Dependencies are pinned in a single `requirements.txt` for reproducible installs (and so the hosted Streamlit app builds from one file). The **scored pipeline imports only the core subset** — pandas, numpy, scikit-learn, joblib, pyarrow — and makes no network calls at run time. The remaining packages (Streamlit, Plotly, OpenAI, python-dotenv) are used only by the interactive demo.

Two execution paths

```

===== SCORED PATH (run.sh) =====
data/*.csv -> generate_features.py -> features.parquet -> predict.py -> output/predictions.csv
              (data_loader + features)                (forecast + model.pkl)

===== DEMO PATH (streamlit) =====
data/*.csv -> app.py -> build_panel -> run_forecast(budget) -> charts + tables
              |
              +--> insights.py -> OpenAI API -> insight card
  
```

The scored path is fully offline and deterministic. The demo path reuses the exact same forecasting core and only adds interactivity + the LLM layer.

Module map (src/)

Module	Responsibility
<code>config.py</code>	canonical schema, campaign-type normalization
<code>data_loader.py</code>	per-platform adapters -> unified long table
<code>features.py</code>	monthly aggregation, lag/rolling/seasonality, partial-month filter
<code>generate_features.py</code>	entry: <code>data/</code> -> <code>features.parquet</code>
<code>model.py</code>	design matrix + three quantile regressors
<code>train.py</code>	offline backtest + train + save <code>pickle/model.pkl</code>
<code>forecast.py</code>	recursive roll-forward, budget simulation, horizon roll-ups
<code>predict.py</code>	entry: <code>features</code> + <code>model</code> -> <code>predictions.csv</code>
<code>insights.py</code>	facts + anomaly rules + LLM explanation (with fallback)
<code>app.py</code> (root)	Streamlit UI

Forecasting pipeline (data flow)

1. **Ingest & unify** — adapters normalize the three platforms into one table.
2. **Aggregate & feature** — daily -> monthly; add lag/rolling/seasonality clues.
3. **Predict** — three quantile models -> revenue ranges; ROAS derived.
4. **Roll forward & up** — recursive months -> 30/60/90 days; channel/type/total.
5. **Explain** (demo only) — grounded facts + anomalies -> LLM insight.

LLM integration workflow

```

forecast + history -> compute_facts() ---+
                                         +--> prompt (facts only) -> OpenAI -> summary
panel -> detect_anomalies() -----+
                                         |
                                         (no key / error) -----+--> rule-based fallback

```

The LLM never sees raw data and is constrained to the computed facts, which keeps insights grounded and avoids hallucinated numbers.